

Lecture7 文件系统 1

问题与概念

1. 问题

1) 怎样理解“文件系统是对磁盘的抽象”这句话？

因为文件在磁盘上,所以管理文件就是管理磁盘——进程是对 CPU 抽象,地址空间是对内存的抽象,文件系统是对磁盘的抽象

2) 从用户角度看文件系统的功能

a) 怎样建立、删除、打开、关闭、读写文件？

b) 怎样确定和修改文件的权限？

3) 从系统的角度看文件系统的实现

a) 怎样管理每个文件？——管理文件描述信息 (fd)

b) 怎样设计并实现文件目录？

c) 怎样管理磁盘空间？

4) 文件系统还要考虑哪些需求？

性能/安全

5) 文件系统与其他操作系统功能的接口

和虚存、进程、IO（最重要）

2. 概念

文件系统	文件	文件分类
文件控制块FCB	文件目录	目录文件
文件系统布局	文件逻辑结构	文件物理结构
文件描述符/文件句柄	FAT/UNIX	文件基本操作
内存结构	文件共享	磁盘空间管理

基本概念

Unix: 万物皆文件, 所有的 IO 设备乃至内核都被视为文件——统一动作: 读/写

1. 文件

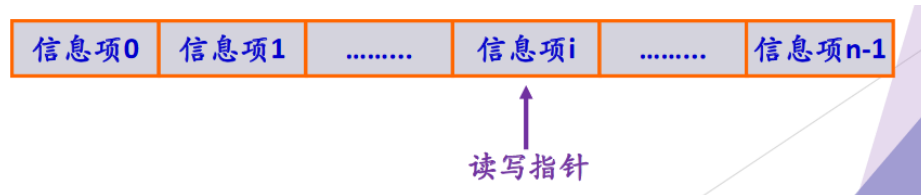
1) 基本概念

- a) 是对磁盘的抽象
- b) 一组带标识(文件名)的、在逻辑上有完整意义的信息项(一个字节或若干字节/页)的序列(强调顺序关系)
 - 信息项: 构成文件内容的基本单位(单个或多个字节), 各信息项之间具有顺序关系

c) ICS——文件是字节序列

2) 文件内容的意义

由文件建立者和使用者解释文件内容的具体含义(操作系统管理文件, 不管文件内容)



3) 文件逻辑结构

文件内部结构: 从用户角度看文件, 由用户的访问模式确定

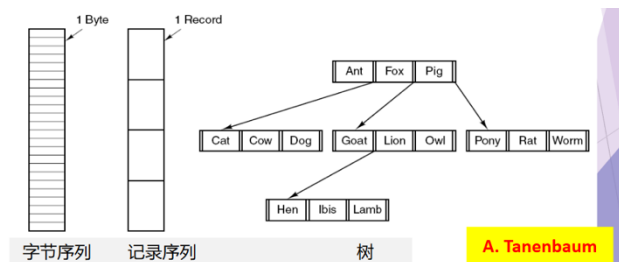
a) 流式文件

数据存储为连续的字节流, 适合顺序访问, 常用于文本文件和二进制文件(用户文件)

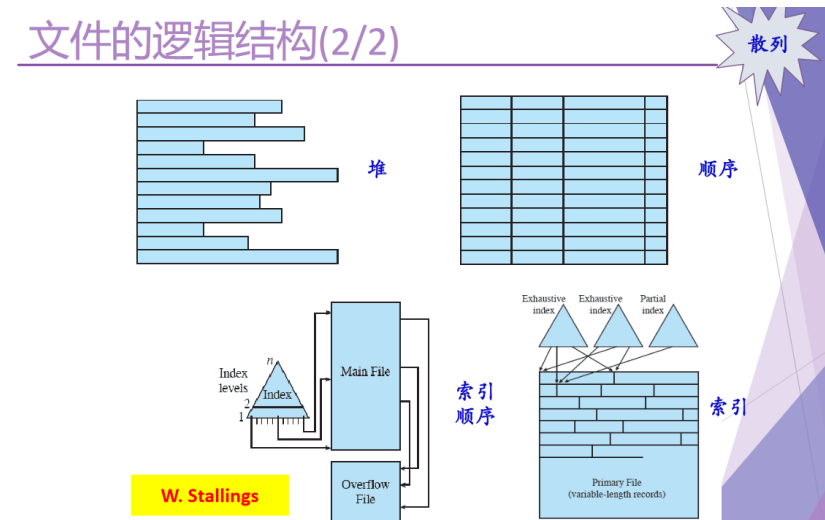
b) 记录式文件

数据组织为记录, 适合随机访问, 常用于数据库和文件系统中的索引文件(目录文件)

操作系统将所有文件分为以上两种; NTFS 目录文件用 B+树组织



文件的逻辑结构(2/2)



4) 文件访问（存取）方式

- 顺序存取（访问）：按字节依次读取
- 随机存取（访问）：从任意位置读写，提供读写位置（e.g., UNIX 的 seek 操作）

5) 文件分类

按照文件性质和用途分（UNIX）

a) 普通文件

- 包含了用户的信息，一般为 ASCII 或二进制文件
- 各种用户可见文件都是普通文件

b) 目录文件：管理文件系统的系统文件

c) 特殊文件（设备文件）

- 字符设备文件：和输入输出有关，用于模仿串行 I/O 设备，例如终端，打印机，网络等；
- 块设备文件：模仿磁盘

d) 其它：管道文件、套接字文件、符号链接文件.....

6) 文件属性

a) 文件控制块（FCB: File Control Block）

- 操作系统为管理文件而设置的数据结构，存放了为管理文件所需的所有有关信息（文件属性/元数据）
- 一个文件对应一个 FCB

- 操作系统中的重要数据结构：中断向量表、PCB、系统调用表、页表、FCB.....

b) 常用属性

文件名，文件号，保护，口令，创建者，当前拥有者，文件地址，文件大小，文件类型，共享计数，创建时间，最后修改时间，最后访问时间，各种标志（只读、隐藏、系统、归档、ASCII/二进制、顺序/随机访问、临时文件、锁）

7) 文件操作

- | | |
|-----------------|-------------------------|
| • create | • append |
| • delete | • seek |
| • open | • get attributes |
| • close | • set attributes |
| • read | • rename |
| • write | |

文件访问模式：进程访问文件数据
前必须先“打开”文件

```
f = open(name, flag);
...
read(f, ...);
...
close(f);
```

seek：修改读写指针

2. 文件系统

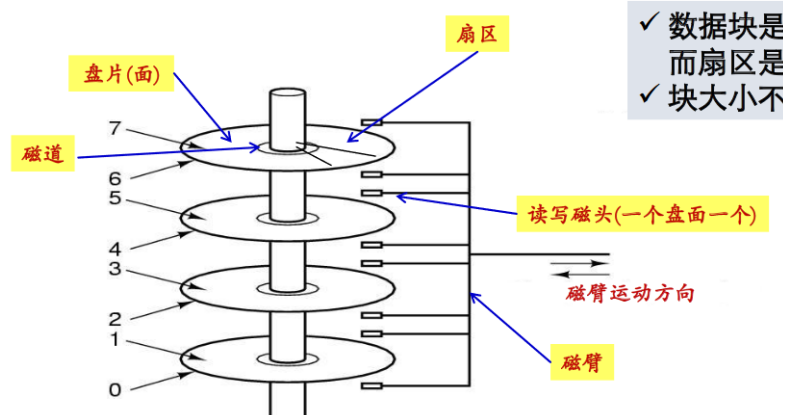
1) 功能：管理持久性数据

统一管理信息资源，管理文件的存储、检索、更新，提供安全可靠的共享和保护手段，并且方便用户使用

- 统一管理磁盘空间，实施磁盘空间的分配与回收
- 实现文件的**按名存取**：将名字空间（文件路径名）映射至磁盘空间（最主要的功能）
- 实现文件信息的共享，并提供数据可靠性和安全保障
- 向用户提供一个方便使用、维护的接口，并向用户提供有关信息
- 提高文件系统的性能（磁盘运行速度相对 CPU 很慢）
- 提供与 I/O 系统的统一接口

3. 存储介质与物理块（磁盘块）

- 1) 典型存储介质：磁盘，固态硬盘、磁带、光盘.....
- 2) 磁盘结构



a) 扇区

- 将磁盘看作一组扇区， $0, 1, \dots, n-1$
- 一般保证扇区写入的原子性（即定义扇区为磁盘的最小存取单元）

b) 物理块（块、数据块、磁盘块）

- 将存储设备划分为大小相等的块，统一编号
- 块是信息存储、传输、分配（即 os 管理磁盘）的基本单位
- Windows 称为“簇”
- 由若干扇区组成

c) 物理块与扇区的区别

- 数据块是逻辑存储单元
- 扇区是物理存储单元
- 块大小不等于扇区大小

d) 物理地址形式

- 磁头号(盘面号)+磁道号(柱面号, 所有磁盘相应磁道组成柱面)
+扇区号
- 扇区：标题（10 字节）+数据（512 字节）+ECC 纠错信息（12-16 字节）

3) 磁盘访问

a) 基本特性

- 任何时刻只有一个磁头处于活动状态
- 输入输出数据流以位串形式出现
- b) 访盘请求的内容
 - 读/写
 - 磁盘地址（设备号+柱面号+磁头号+扇区号）
 - 内存地址（源/目）
- c) 过程
 - 寻道时间：磁头移动定位到指定磁道（最耗费时间）
 - 旋转延迟时间：等待指定扇区从磁头下旋转经过（比较耗费时间）
 - 数据传输时间：数据在磁盘与内存之间的实际传输

SSD 不需要寻道和旋转，因此比磁盘快得多

4. 文件目录、目录项与目录文件

1) 文件目录

- a) 是文件控制块的有序集合（一个抽象的概念，以目录文件的形式存在）
- b) 统一管理每个文件的信息
- c) 将所有文件的管理信息组织（如何组织由 os 决定）在一起，即构成文件目录

2) 目录文件（os 创建）

- a) 将文件目录以文件的形式存放在磁盘上
- b) 一种特殊类型的文件，其内容由目录项构成

3) 目录项

- a) 构成文件目录的基本单元
- b) 目录项可以是 FCB（也可以优化为 FCB 的子集）

4) 注意

- a) 为确保映射的完整性，仅允许内核修改目录
- b) 应用程序通过系统调用访问目录

5) 树形目录结构

提高文件系统性能的一种手段

a) 路径名（文件名）

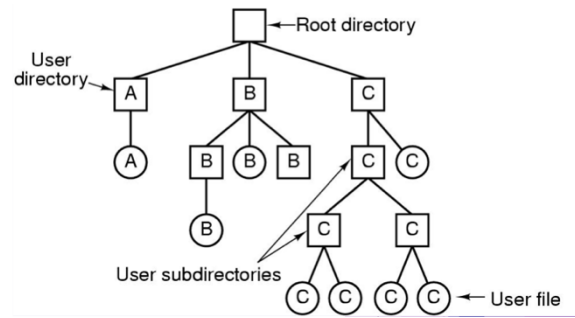
- 绝对路径：从根目录开始
- 相对路径：从当前目录开始

b) 当前目录/工作目录

- 每个进程一个，可以改变，用于解析文件名
- 允许用户使用相对路径名代替绝对路径名

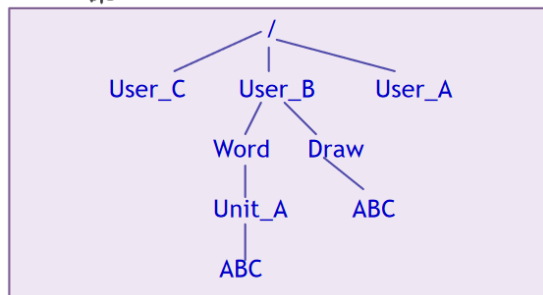
c) 目录操作

- create, delete
- opendir, closedir
- readdir, rename
- link, unlink
- 典型目录操作
 - ✓ 创建/删除目录文件
 - ✓ 列目录
 - ✓ 遍历路径
 - ✓

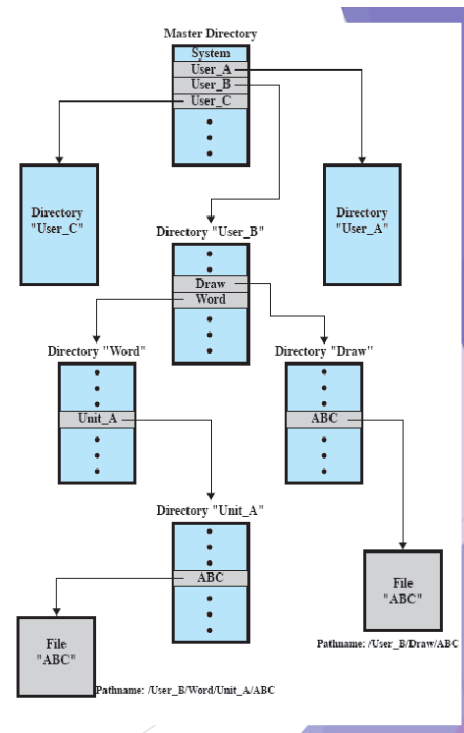


目录文件示例

- 试画出右图中对应的文件目录的树形结构
- 右图中有多少个目录文件？
- 试给出解析文件名 /User_B/Draw/ABC 的步骤



目录文件：文件夹、子目录...由目录项组成



文件系统的实现 (1)

1. 概述

- 1) 实现文件系统需要考虑磁盘与内存中的内容布局(打开文件: 文件与内存建立联系)
- 2) 问题
 - a) 磁盘上
 - 如何启动所存储的操作系统?
 - 磁盘是怎样管理的? 即怎样获取磁盘的有关信息?
 - 目录文件在磁盘上怎么存放? 普通文件在磁盘上怎么存放?
 - b) 内存中
 - 进程使用文件时, 操作系统如何支持与管理? (进程通过 os 打开文件, 之后通过 os 进行操作)

2. 相关术语

- 1) 磁盘分区(partition)

把一个物理磁盘的存储空间划分为几个相互独立的部分, 成为分区(也可以不分区)
- 2) 文件卷(volume)

逻辑分区, 由一个或多个物理块(簇)组成

 - a) 一个文件卷可以是整个磁盘或部分磁盘或跨盘(RAID)
 - b) 同一个文件卷中使用同一份管理数据进行文件分配和磁盘空闲空间管理, 不同的文件卷中的管理数据相互独立
 - c) 一个文件卷上的内容
 - 文件系统信息
 - 一组文件(用户文件和目录文件)
 - 未分配空间
 - d) 物理块/块/簇

一个或多个(2的幂)连续的扇区, 可寻址数据块
- 3) 格式化(format)

在一个文件卷上建立文件系统(元数据)的过程, 即建立并初始化用于文

件分配和磁盘空闲空间管理的管理数据

3. 磁盘上文件系统的内容

1) 引导区

- a) 包括从该卷引导操作系统所需要的信息
- b) 每个卷（分区）一个，通常为第一个扇区

2) 卷（分区）信息

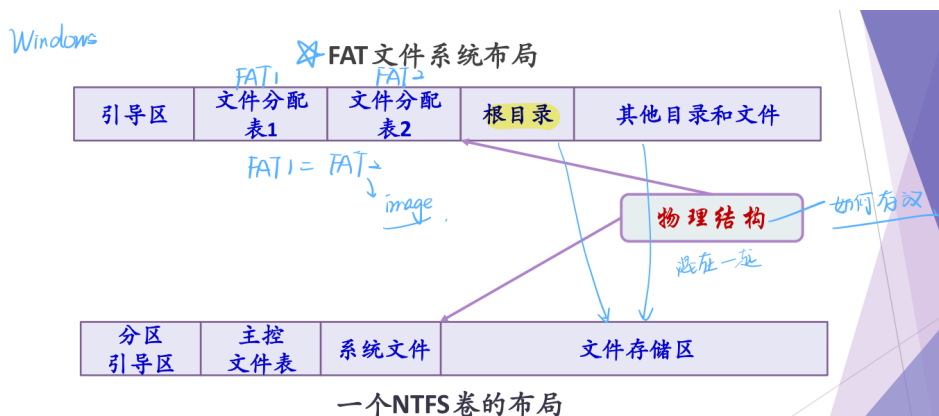
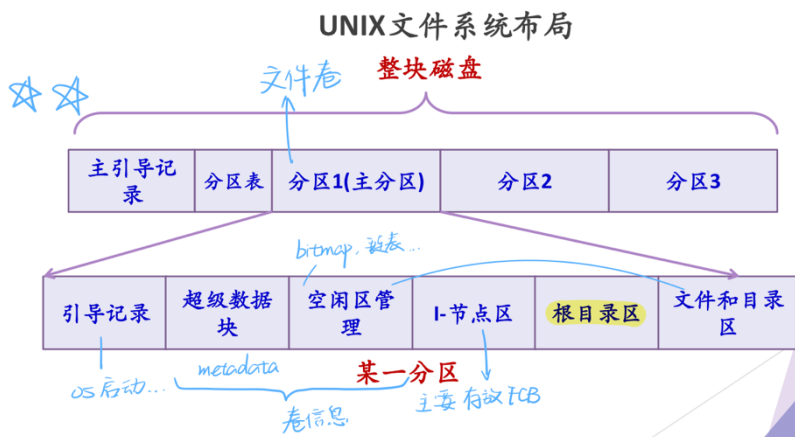
- a) 块（簇）数
- b) 块（簇）大小
- c) 空闲块（簇）数量和指针
- d) 空闲 FCB 数量和指针

回顾：物理块（块 block、簇 cluster）：信息存储、传输、分配的独立单位；存储设备划分为大小相等的物理块，统一编号

3) 目录文件

4) 用户文件

4. 磁盘上文件系统的布局



FAT 文件系统中文件分配表 2 是备份文件分配表，用于在 FAT1 损坏时进行恢复

5. 文件的物理结构

1) 简介

a) 内容

- 文件在物理介质上的存放方式
- 系统分配给文件的物理块的位置和顺序

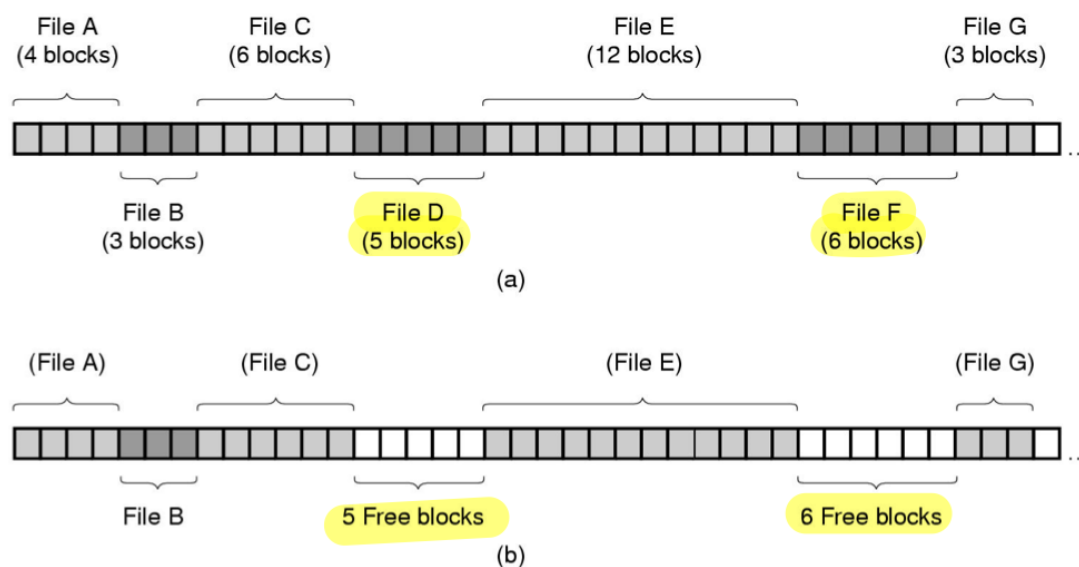
b) 要考虑的问题

- 存储效率（外部碎片等）
- 读写性能（访问速度）

2) 连续结构（顺序）

a) 简介

文件的信息存放在若干连续物理块中



提问：在 FCB 中如何记录文件的磁盘地址？只需要记录文件所在块的首块地址和文件大小即可

b) 优点

- 简单、高效
- 支持顺序存取和随机存取
- 所需的磁盘寻道次数和寻道时间最少
- 可以同时读入多个块，检索一个块也很容易

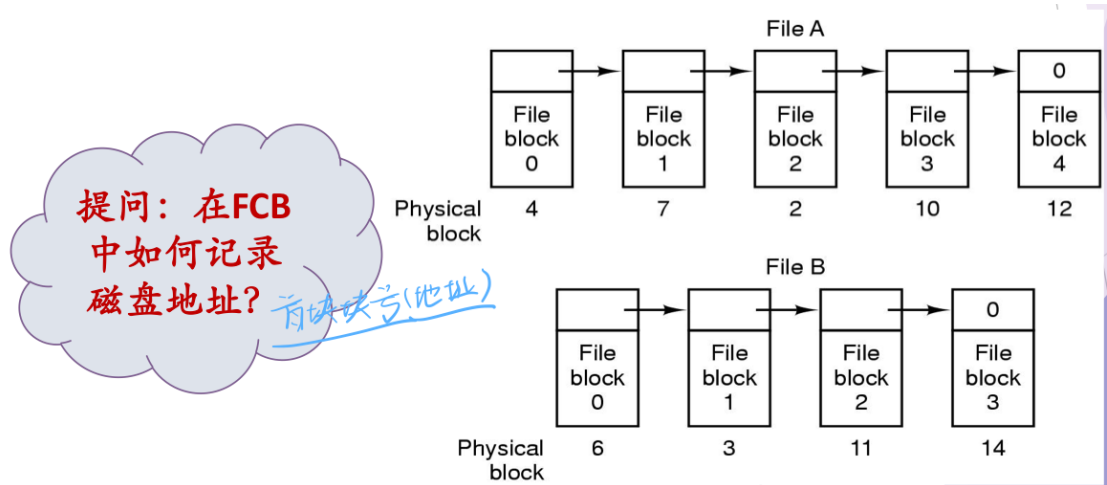
c) 缺点

- 文件不能动态增长
 - ✓ 预留空间（浪费）
 - ✓ 重新分配和移动文件

3) 链接结构

a) 简介

一个文件的信息存放在若干不连续的物理块中，各块之间通过指针连接，前一个物理块指向下一个物理块



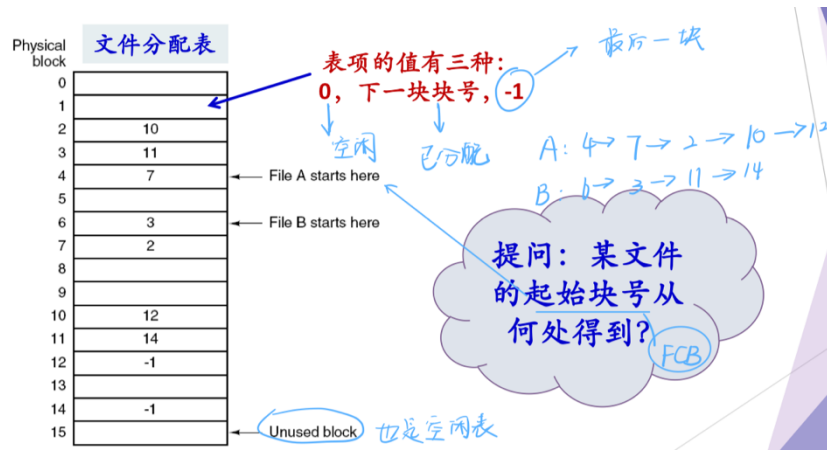
b) 优点

- 提高了磁盘空间利用率，不存在外部碎片问题
- 有利于文件内容的插入和删除
- 有利于文件动态扩充

c) 缺点

- 存取速度慢，不适于随机存取（只能从头遍历文件的块进行查找）
- 可靠性问题，如链接指针出错
- 更多的寻道次数和寻道时间
- 链接指针占用一定的空间（容易破坏 2^k 的大小，2 的幂的大小易处理）

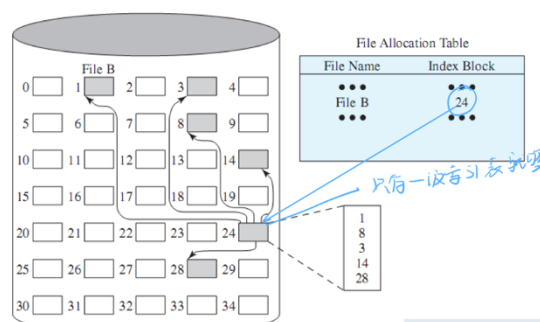
链接结构的一个变形：文件分配表 FAT



4) 索引结构

a) 简介

- 一个文件的信息存放在若干不连续物理块中
- 系统为每个文件建立一个专用数据结构：索引表，并将这些块的块号存放在一个索引表中
- 一个索引表就是磁盘块地址数组，其中第 i 个条目指向文件的第 i 块



b) 提问

- 索引表存放在何处？
 - ✓ 磁盘的专用区域
 - ✓ 离散化存储
- 在 FCB 中如何记录磁盘地址
 - ✓ 记录索引表地址即可

c) 优点

- 保持了链接结构的优点，又解决了其缺点
- 既能顺序存取，又能随机存取
- 满足了文件动态增长、插入、删除的要求
- 充分利用磁盘空间

d) 缺点

- 较多的寻道次数和寻道时间
- 索引表本身带来系统开销 (e.g., 内存、磁盘空间, 存取时间)

e) 索引表组织

问题: 索引表很大, 需要多个物理块存放时怎么办?

- 链接方式

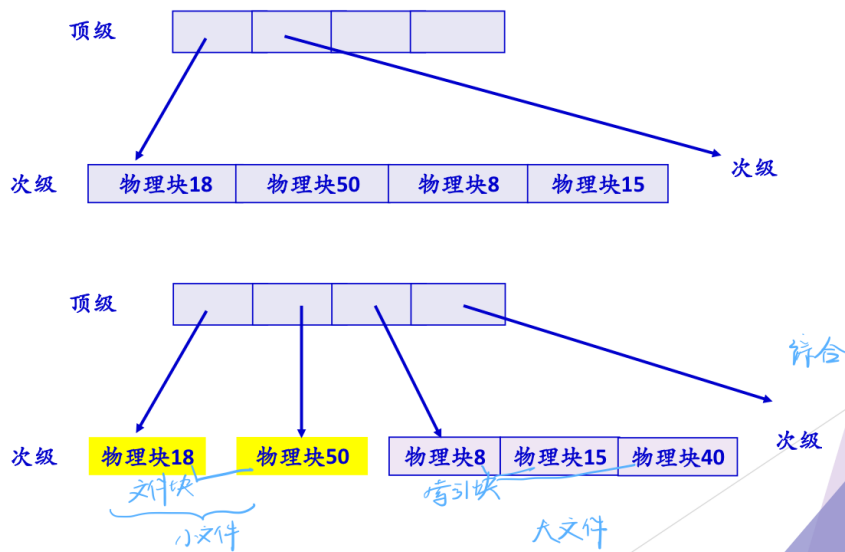
一个盘块存一个索引表, 多个索引表链接起来

- 多级索引方式

将文件的索引表 (二级索引) 的地址存放在另一个索引表 (一级索引) 中

- 综合模式

直接索引与间接索引结合, 更加灵活

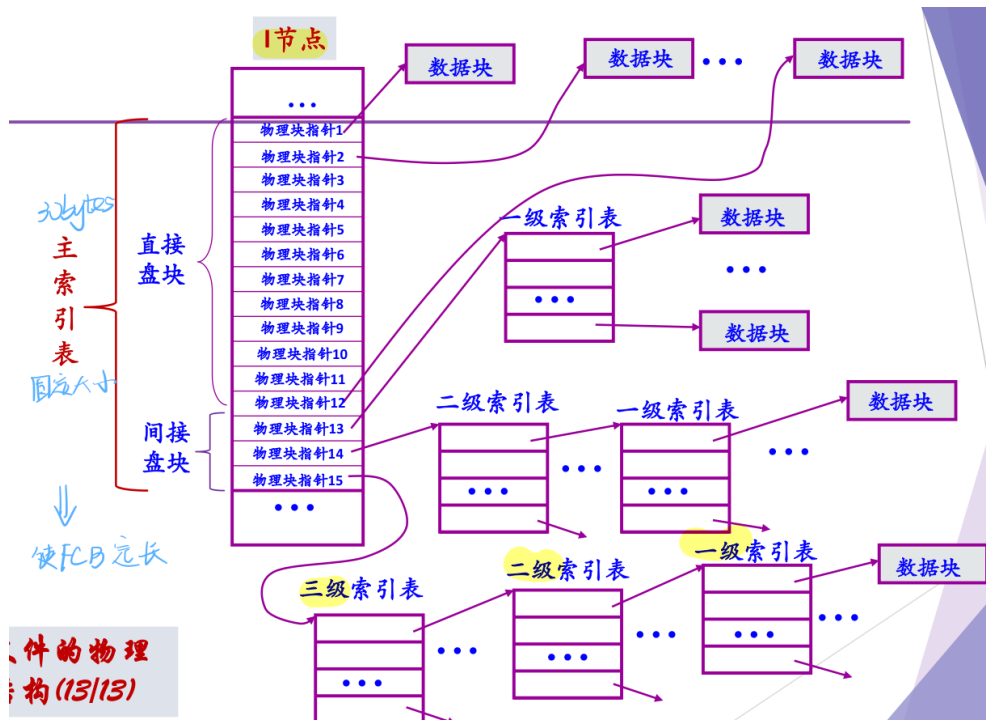


UNIX文件系统采用的是多级索引结构(综合模式)

- ▶ 每个文件的索引表有15个索引项, 每项2个字节 (16 bits)
- ▶ 最前面12项直接登记存放文件信息的物理块号 (直接寻址)
- ▶ 如果文件大于12块, 则利用第13项指向一个物理块, 该块中最多可放256个文件物理块的块号 (一级索引表)
- ▶ 对于更大的文件还可利用第14和第15项作为二级和三级索引表

UNIX中采用了三级索引结构后, 文件最大可达到? 个物理块

假设扇区大小为512字节, 物理块等于扇区大小, 一级索引表可以存放256个物理块号



关键问题：在 FCB 中如何记录磁盘地址？

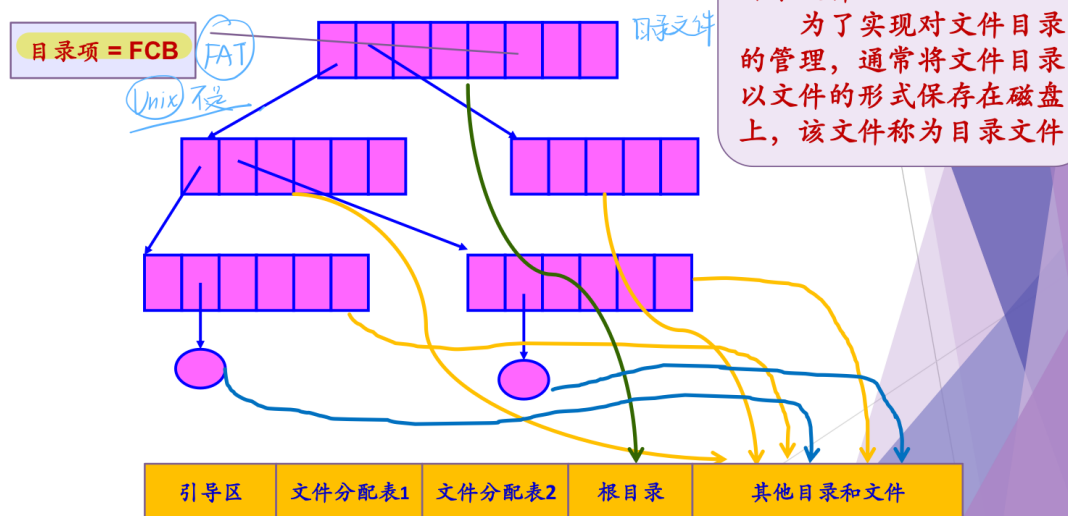
顺序：记录首物理块块号+文件长度/块数

链接：记录首物理块地址

索引：记录索引表地址

6. 目录文件的组织方式

3. 目录文件的组织方式(1/2)



1) 顺序表（查找慢）

2) 散列表

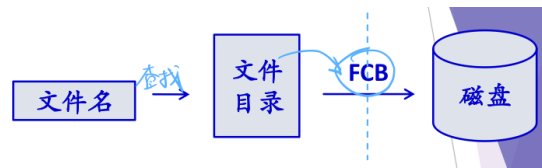
- a) 将目录项组织成一散列表
- b) 根据文件名计算散列值，得到一个指向表中文件的指针(冲突值用链表组织)
- c) 优点：查找速度快

3) B 树/B+树

NTFS 文件系统中目录文件采用 B+树组织目录项

7. 文件目录检索（访问一个文件）

1) 目录检索



- a) 用户给出文件名->按文件名
查找目录项/FCB
- b) 文件名解析：把逻辑名字转换成物理资源
路径名
 - 全路径名：从根开始 \A\B\C\File1
 - 相对路径：从当前目录开始 C\File1

2) 文件寻址

根据目录项/FCB 中文件物理地址等信息，计算文件中任意记录或字符在存取介质上的地址

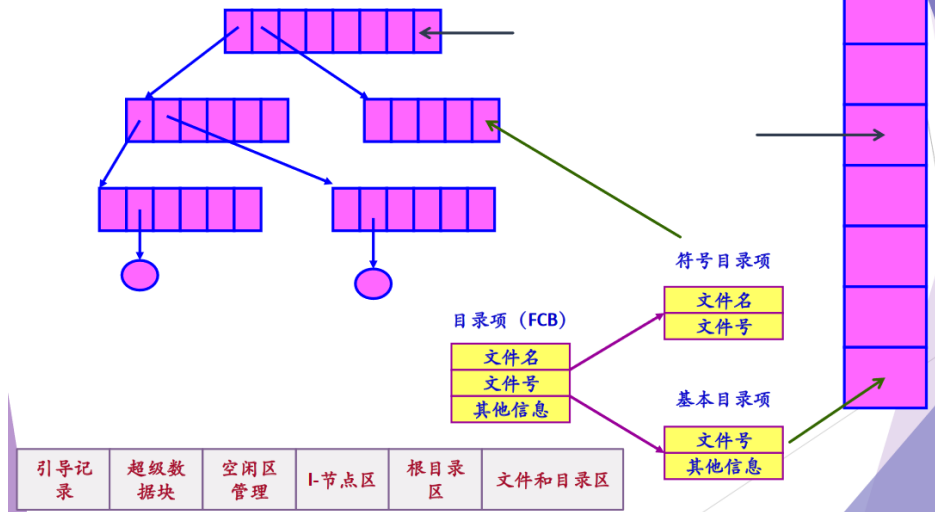
问题：如何加快目录检索？

3) 目录文件的改进

- a) 目录项分解法：将 FCB 分成两部分
 - 符号目录项：文件名、文件号
用于构成目录文件
 - 基本目录项：除文件名外的所有字段
用于构成 FCB

UNIX 的 I 节点（索引节点或 inode）即为此方法

目录文件的改进(2/3)



目录文件的改进(3/3)

例子:

假设一个FCB有 48 个字节

物理块大小 512 字节 10 FCB/块

符号目录项占 8 字节 (文件名6字节, 文件号2字节)

基本目录项占 $48 - 6 = 42$ 字节

分解前: 占13块

分解后: 符号文件占2块
基本文件占11块

假设一个目录文件有128个目录项, 计算查找一个文件的平均访盘次数?

分解前: 7次 平均找7块

分解后: 2.5次 $\frac{1+7}{2} = 2.5$

文件目录改进后减少了访问硬盘的次数, 提高了检索速度

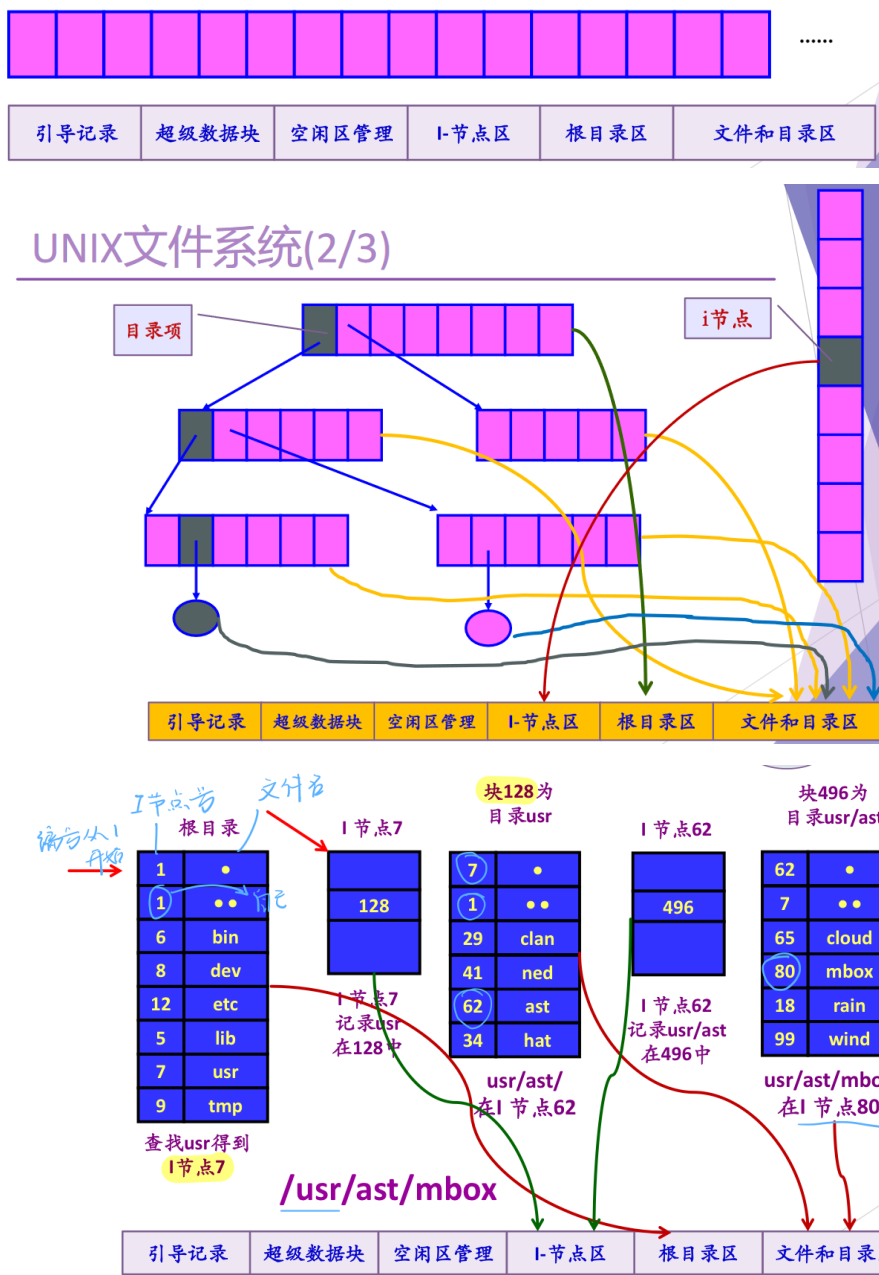
当前目录

文件系统实例

1. UNIX 文件系统

1) 简介

- a) $FCB = \text{目录项} + i \text{节点}$
- b) $\text{目录项} = \text{文件名} + i \text{节点号}$
- c) 目录文件由目录项构成
- d) $i \text{节点}$: 描述文件的相关信息
- e) 每个文件由一个目录项、一个 $i \text{节点}$ 和若干磁盘块构成



2. Windows: FAT16 文件系统

1) 简介

- ▶ **簇**: 1、2、4、8、16、32或64扇区/簇
- ▶ 文件系统的数据记录在“**引导扇区**”中
- ▶ **文件分配表FAT**的作用
描述簇的分配状态、标注下一簇的簇号
- ▶ **FAT表项**: 占2字节
- ▶ **目录项**: 占32字节 = FCB
- ▶ **根目录大小固定** FAT16
现在不定

1.5 byte
2 byte FAT16
4 byte
FAT12、FAT16、FAT32

引导区	文件分配表 1	文件分配表 2	根目录 文件	其他目录文件和 普通文件
-----	------------	------------	-----------	-----------------

2) 主引导记录扇区 MBR: 0 号扇区

字节偏移量 (16进制)	域长	含义
00 - 1BD	446字节	引导代码
1BE - 1CD	16字节	分区表项1
1CE - 1DD	16字节	分区表项2
1DE - 1ED	16字节	分区表项3
1EE - 1FD	16字节	分区表项4
1FE - 1FF	2字节	扇区结束标记 (55AA)

MBR	分区表	分区1(主分区)	分区2	分区3
-----	-----	----------	-----	-----

3) 分区引导扇区 DBR

FAT文件系统——DBR

FAT32

引导区	文件分配表1	文件分配表2	根目录	其他目录和文件
-----	--------	--------	-----	---------

字节偏移量 (16进制)	域长	样值 (16进制)	含义
00	3字节	EB 3C 90	转移指令
03	8字节	MSDOS5.0	文件系统标志(ASCII码)
0B	25字节		BIOS参数块 (BIOS Parameter Block, BPB)
24	54字节		扩展BIOS参数块(Extended BIOS Parameter Block, EBPB)
5A	410字节		引导代码 跳转...
1FE	2字节	55 AA	扇区结束标记

a) BIOS 参数块

引导扇区 - BIOS参数块 *5 bytes*

FAT32

字节偏移量 (16进制)	域长	样值(16进制)	含义
0B	2字节	00 02	每扇区字节数
→ 0D	1字节	08	每簇扇区数
0E	2字节	01 00	保留扇区数: 从分区引导扇区到第一个文件分配表开始的扇区数
10	1字节	02	文件分配表个数 <i>第2个是第二个副本</i>
→ 11	2字节	00 02	根目录项数 <i>FAT32 不用(!)</i>
13	2字节	00 00	扇区数(小): 卷上的扇区数, 如果该数适合于16位(65535)的话
15	1字节	F8	介质类型: F8表明为硬盘, F0表明为软盘
16	2字节	C9 00	每个文件分配表的扇区数(FAT32不用)
18	2字节	3F 00	每磁道扇区数
1A	2字节	10 00	磁头数
1C	4字节	3F 00 00 00	隐藏扇区数
20	4字节	51 42 06 00	扇区数(大): 如果小扇区数域的取值为0, 该域包含的是卷中的扇区总数

b) 扩展 BIOS 参数块 (EBPB)

引导扇区 - 扩展BIOS参数块(EBPB) *54 bytes*

FAT32

字节偏移量 (16进制)	域长	含义
→ 24	4字节	每个文件分配表的扇区数(FAT32用)
28	2字节	标记: bit7为1, 表示只有一个FAT; 否则, 两个FAT互为镜像
2A	2字节	版本号
→ 2C	2字节	根目录起始簇号, 通常为2 <i>从2开始.</i>
30	2字节	FSINFO所在扇区, 通常1扇区
32	2字节	引导扇区备份, 通常是6号扇区
34	12字节	未用
40	1字节	BIOS Int 13H设备号
41	1字节	未用
42	1字节	扩展引导标识, 如果后面三个值有效, 设为0x29
43	4字节	卷序列号: 当格式化卷时创建的一个唯一的数字
47	11字节	卷标(ASCII码), 建立文件系统时由用户指定
52	8字节	系统ID: 根据磁盘的格式, 该域的取值为FAT12或FAT16

4) 文件分配表 FAT

a) 简介

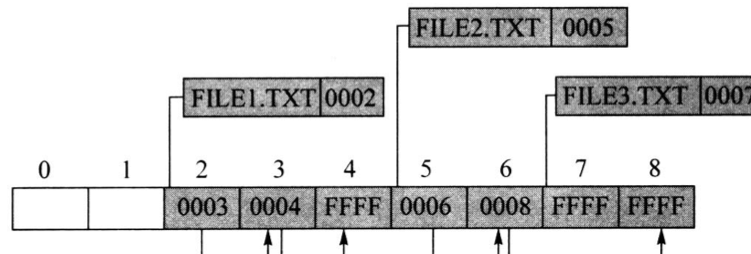
可以看作一个整数数组, 每个整数代表数据区的一个簇号

b) 状态

- 未使用
- 坏簇

- 系统保留
- 被文件占用（下一簇簇号）
- 最后一簇（0xFFFF）

c) 簇号从 0 开始编号，簇 0 和簇 1 保留（从 2 开始放置）



d) FAT16 目录项

偏移	域长 <i>bytes</i>	含义
00h	8	文件名
08h	3	文件扩展名
0Bh	1	文件属性字节
0Ch	10	保留
16h	2	最后一次修改的时间
18h	2	最后一次修改的日期
1Ah	2	起始簇号
1Ch	4	文件大小

位	7-6	5	4	3	2	1	0
	保留	归档	目录	卷标	系统	隐藏	只读

Handwritten notes: A bracket groups the last four fields (16h-1Ch) with the label "bytes". A red arrow points from the "起始簇号" field to the "位" table. A blue circle labeled "FAT" has an arrow pointing to the "起始簇号" field.

3. Windows: FAT32 文件系统

1) 简介

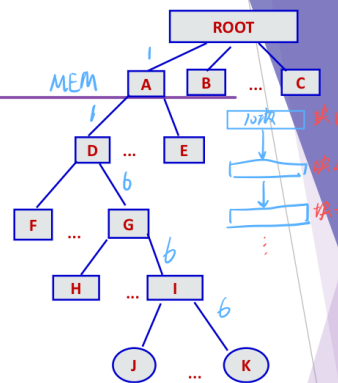
- FAT32 的根目录区（ROOT 区，根目录文件）不是固定区域、固定大小，而是数据区的一部分，采用与子目录文件相同的管理方式
- 目录项仍占 32 字节，但分为各种类型（包括：“.”目录项、“..”目录项、短文件名目录项、卷标项（根目录）、已删除目录项（第一字节为 0xE5）、长文件名目录项等）
- 支持长文件名格式（255 个 char）
- 支持 Unicode（2bytes/char）
- 无法支持高级容错特性，不具有内部安全性

启动磁盘次数 = 访问
有连接块 找K: 5 ~ 20
练习1 又数 访问K: (1) 1 ~ 4
6块

有一个文件系统，根目录常驻内存，如图所示。
目录文件采用链接结构，规定一个目录下最多
存放60个下级文件。下级文件可以是目录文件，也
可以是普通文件。每个磁盘块可存放10个下级文件
的目录项，若下级文件为目录文件，则目录项给出
该目录文件的第一块地址，否则给出普通文件的
FCB的地址。

假设文件按自左向右的顺序建立，...表示有若干内容未显示。

(1) 假设普通文件采用UNIX的三级索引结构，即FCB中给出13个磁盘地址，前10个磁盘地址指出文件前10块的物理地址，第11个磁盘地址指向一级索引表（给出256个磁盘地址）；第12个磁盘地址指向二级索引表，二级索引表中指出256个一级索引表的地址；第13个磁盘地址指向三级索引表，三级索引表中指出256个二级索引表的地址。若要读文件A\D\G\I\K中的某一块，最少要启动磁盘几次？最多要启动磁盘几次？



(2) 若普通文件采用链接结构，要读A\D\G\I\K的第55块，最少启动硬盘几次？最多几次？

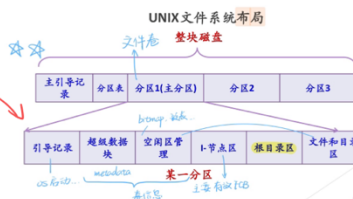
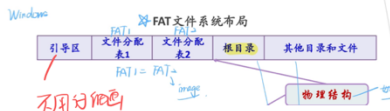
(3) 若普通文件采用顺序结构，要读A\D\G\I\K的第5555块，最少启动硬盘几次？最多几次？

练习2

extremely important (just for exam...)

假设一块刚格式化好的磁盘大小为2M，每块/簇为512字节；要求画出UNIX和FAT16文件系统布局

- ▶ \ mkdir A
- ▶ A mkdir B
- ▶ B create File1(4块/簇)
- ▶ \ mkdir C
- ▶ \ mkdir D
- ▶ C mkdir E
- ▶ E create File2(16块/簇)
- ▶ E mkdir F
- ▶ F create File3(8块/簇)
- ▶ F create File4(2块/簇)



课堂练习 4.25 刘冲雨 2100012289

	最少读盘次数	最多读盘次数
① 在根目录文件中查找A的目录项	0	0
② 启动磁盘, 将A读入内存, 查找D的目录项 <边读边查找>	1	1
③ 启动磁盘, 将D读入内存, 查找G的目录项	1	6
④ 启动磁盘, 将G读入内存, 查找I的目录项	1	6
⑤ 启动磁盘, 将I读入内存, 查找K的目录项	1	6
⑥ 启动磁盘, 将K的FCB读入内存	1	1
	5	20

	最少读盘次数	最多 ~
(1) 三级索引结构	1	4
(2) 链接结构	55	55
(3) 顺序结构	1	1

2. FAT16: 2B/entry

磁盘大小: 2MB = 2^{21} B

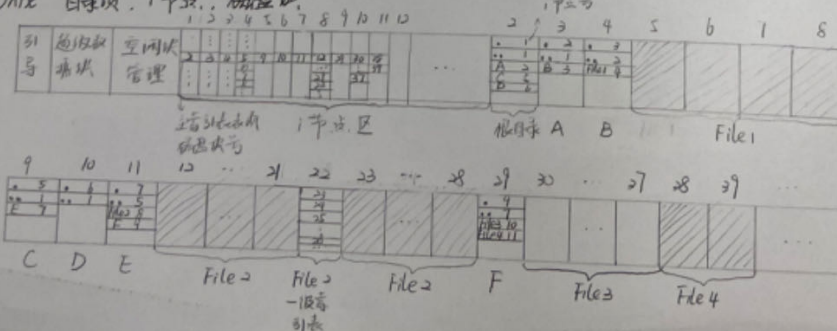
每簇大小: 2^9 B $\Leftrightarrow$ 2^{12} 簇

$\Rightarrow$ FAT表: 2^{13} B $\Leftrightarrow$ 2^9 簇

$512B/32B/16 = 16簇/块$



Unix 目录项, 1字节, 磁盘块



4. NTFS 文件系统

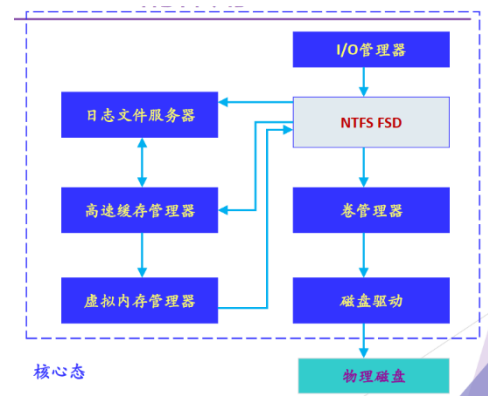
1) 设计思想

设计目标：可靠、高效、安全

- a) 为了满足可靠数据存储和访问的需求，NTFS 提供了基于原子事务概念（Atomic transaction）的文件系统可恢复性
- b) NTFS 对关键文件系统信息采用冗余存储（类似于 FAT1 和 FAT2）
- c) NTFS 提供了综合的安全模型（对象模型）以及支持加密文件系统（EFS, Encrypted File System），可以阻止非授权用户访问加密文件
- d) NTFS 为改进的多级目录结构，支持文件别名（FAT 不支持）
- e) NTFS 文件由多个文件属性构成，每个属性由属性名和属性流（stream，简单字节队列）组成（NTFS 文件属性可以和数据放在一起（如果数据较小的话））
- f) NTFS 文件支持数据压缩功能
- g) NTFS 卷结构支持容错功能

2) NTFS FSD（文件系统驱动程序）

- a) FSD 是一种内核模式驱动程序，其主要作用是使操作系统能够与存储设备上的 NTFS 卷进行交互。它负责解释 NTFS 格式的数据结构，并执行文件系统操作
- b) Windows 中文件系统和 IO 密不可分



3) NTFS 磁盘结构

- a) 卷
 - 建立在磁盘分区上
 - 一个磁盘可以有多个卷，一个卷也可以跨多个磁盘
 - （已格式化的）卷上的数据可分为：元数据和用户数据
- b) 簇
 - 磁盘空间分配和回收的基本单位
 - 簇大小是用户在使用 Format 命令或其他的格式化程序格式化卷

时确定的

- NTFS 使用 LCN (Logical Cluster Number, 逻辑簇号) 和 VCN (Virtual Cluster Number, 虚拟簇号) 来进行簇的定位
 - ✓ LCN 是对整个卷中所有的簇从头到尾所进行的简单编号
 - ✓ VCN 是对属于特定文件的簇从头到尾进行编号, 以便于引用文件中的数据

c) NTFS 分区和簇大小

卷大小	每簇的扇区数	簇大小
<=512MB	1	512B
512MB~1GB	2	1KB
1GB~2GB	4	2KB
2GB~4GB	8	4KB
4GB~8GB	16	8KB
8GB~16GB	32	16KB
16GB~32GB	64	32KB
>32GB	128	64KB

NTFS一卷最大为 2^{64} 字节

4) NTFS 文件组织

a) 文件名称

- NTFS 路径名中的每个文件名/目录名的长度可达 255 个字节, 可以包含 Unicode 字符、多个空格及句点
- NTFS 卷上的每个文件都有一个 64 位的, 称为文件引用号的唯一标识
- 文件引用号的组成
 - ✓ 文件号 (48 位, 该文件在 MFT 中的位置)
 - ✓ 文件顺序号

b) 主控文件表

- MFT (Master File Table, 主控文件表)
NTFS 卷结构的核心, 是 NTFS 中最重要的系统文件, 包含了卷中所有文件的信息
- MFT 是以文件记录数组来实现的
 - ✓ 每一个记录描述了卷中的一个文件或目录 (例如: 1K 大小)
 - ✓ MFT 自身也是一个文件 (也有一个记录)
 - ✓ 文件系统中每一个文件或目录都至少有一个 MFT 项

- ✓ 一般：每个 MFT 项的前几十个字节有固定的头结构，用以描述本 MFT 项的相关信息；之后的字节用于存放属性（属性名，属性值）；文件或目录的其余信息可放到卷中其他可用簇中
- 访问文件：首先在 MFT 中找到对应的 MFT 项，再根据 MFT 项记录的信息找到文件内容



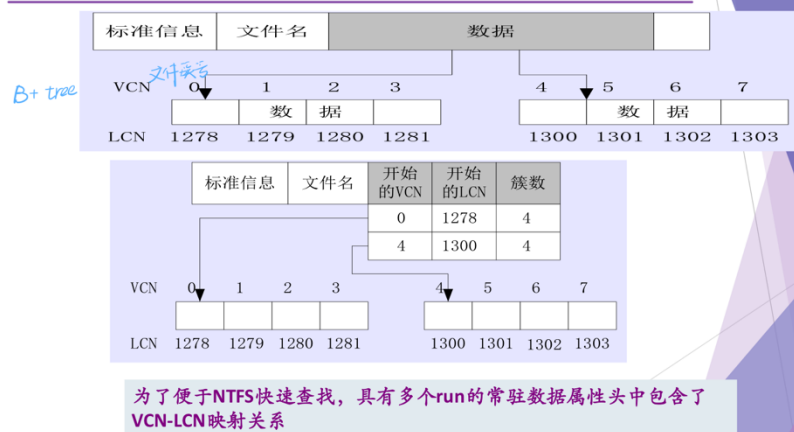
大文件的MFT

MFT记录扩展

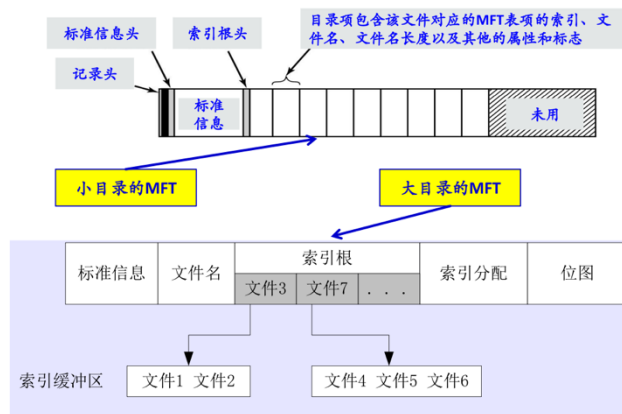
c) 属性

- NTFS 中，所有信息都被称为属性
 - ✓ NTFS 文件是属性/属性值的集合，通常所说的文件内容是指未命名数据属性流
 - ✓ 每个属性由单个的流(stream)组成
- NTFS 提供对属性的各种操作
 - ✓ 读/写操作一般是针对文件的未命名属性的
 - ✓ 创建、删除、读取（字节范围）以及写入（字节范围）
- 常驻属性与非常驻属性
 - ✓ 标准信息、文件名、索引根（根目录）是常驻属性
 - ✓ 延展(run)或延伸(extent): MFT 之外的区域，可存放变长的非常驻属性

一般文件的常驻属性与非常驻属性



目录的常驻属性与非常驻属性



- NTFS 卷上文件的常用属性

NTFS卷上文件的常用属性

属性	描述
标准信息	标志位，时间戳等
文件名	Unicode 文件名
安全描述符	废弃。安全信息现在用\$Extend \$Secure表示
属性列表	额外的MFT记录的位置，如果需要的话
对象ID	对此卷唯一的64位文件标识符
再解析点	用于加载和符号链接
卷名	当前卷的名字（仅用于\$Volume）
卷信息	卷版本（仅用于\$Volume）
索引根	用于目录
索引分配	用于很大的目录
位图	\$MFT文件及索引的位图
日志工具流	控制记录日志到\$LogFile
数据	数据流；可以重复

d) NTFS 元数据文件

MFT 的前 16 个项是为元数据文件保留的，之后则是普通的用户文件和目录

- 0: \$Mft: MFT 本身
- 1: \$MftMirr: MFT 镜像
- 2: \$LogFile: 日志文件
- 3: \$Volume: 卷文件
- 4: \$AttrDef: 属性定义表
- 5: \$\: 根目录文件
- 6: \$Bitmap: 位图文件
- 7: \$Boot: 引导文件
- 8: \$BadClus: 坏簇文件
- 9: \$Secure: 安全文件
- 10: \$UpCase: 大小写字符转换表文件
- 11: \$Extended metadata directory: 扩展元数据目录
- 12, 13, 14, 15:
- >15: 其他用户文件和目录

e) NTFS 目录组织与索引

- 将目录中的文件名和子目录名进行排序,保存在索引根属性中(文件目录是文件名的索引,还包括文件引用、文件时间和大小等信息)
- 对于一个大目录,经排序的文件(目录)名实际存储在 4KB 大小的索引缓冲区中(索引缓冲区是通过 B+树组织的)
- 索引分配属性包含了索引缓冲区的 VCN 到 LCN 映射
- 位图属性跟踪在索引缓冲区中哪些 VCN 是已使用而哪些是空闲的

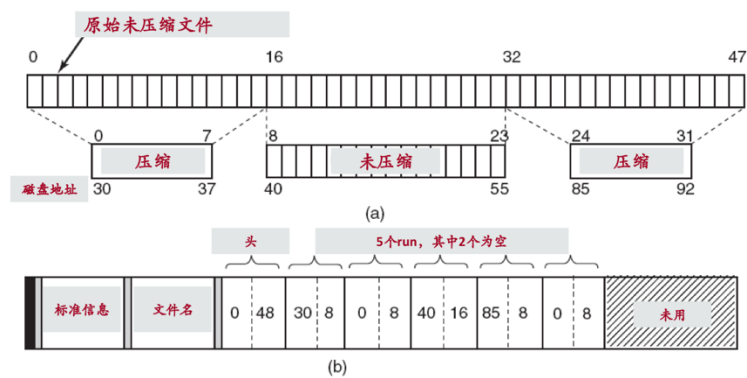
5) NTFS 数据压缩

a) 简介

- 数据压缩是 NTFS 文件系统的一个重要特征
- NTFS 压缩功能可以对单个文件、整个目录或 NTFS 卷上的整个目录树进行压缩
- NTFS 压缩只在用户数据上进行,而不能在文件系统元数据上进行
- 数据压缩可以减少磁盘使用空间,但是由于每次解压缩需要大量的数据运算,使用压缩功能将会导致 NTFS 卷的性能下降(如果要拷贝一个压缩文件,其过程是:解压缩、拷贝、重新对拷贝的文件进行压缩,这些都大大增加了 CPU 的处理时间)

b) 压缩过程

- NTFS 是以 16 个簇为压缩单元进行一般文件的压缩
- NTFS 决定压缩这 16 个簇后是否能腾出一个簇,如果能则只分配所需的簇数,并将数据写到磁盘上;否则直接将数据写到磁盘上
- 当 NTFS 向压缩文件写数据时,它确保每个 run 都以一个虚拟 16 簇边界开始
- 每个 run 中 VCN 都是以 16 的倍数开始的,并且 run 的长度不大于 16



NTFS 可恢复性

通过日志记录来实现

子操作在磁盘运行之前，记录在日志文件中

系统恢复阶段，NTFS 根据日志文件中的文件操作信息，对部分完成的事务进行重做或者撤销，保证磁盘文件系统的一致性（预写日志记录）

6) NTFS 可恢复性

a) 通过日志记录来实现

b) 子操作在磁盘运行之前，记录在日志文件中

c) 系统恢复阶段，NTFS 根据日志文件中的文件操作信息，对部分完成的事务进行重做或者撤销，保证磁盘文件系统的一致性（预写日志记录）

文件系统的实现 (2)

1. 磁盘空间管理

1) 位图法

- 用一串二进制位反映磁盘空间中分配使用情况, 每个物理块对应一位, 分配物理块为 0, 否则为 1 (空闲)
- 申请物理块时, 可以在位示图中查找为 1 的位, 返回对应物理块号
- 归还时, 将对应位转置 1

已知块号, 则磁盘地址:

柱面号 = $\lfloor \text{块号} / (\text{磁头数} \times \text{扇区数}) \rfloor$
磁头号 = $\lfloor (\text{块号} \bmod (\text{磁头数} \times \text{扇区数})) / \text{扇区数} \rfloor$
扇区号 = $(\text{块号} \bmod (\text{磁头数} \times \text{扇区数})) \bmod \text{扇区数}$

已知磁盘地址:

块号 = 柱面号 $\times$ (磁头数 $\times$ 扇区数) + 磁头号 $\times$ 扇区数 + 扇区号

位图计算公式:

已知字号 i , 位号 j : 块号 = $i \times \text{字长} + j$

已知块号: 字号 = $\lfloor \text{块号} / \text{字长} \rfloor$ 位号 = 块号 $\bmod$ 字长

2) 空闲链表

将所有空闲块记录在一个表中, 即空闲块表, 有两项 (首块地址+文件长度)

3) 空闲块链表

- 把所有空闲块链成一个链
- 缺点: 块很可能离散地分布在磁盘上
- 扩展 (改进): 成组链接法

• UNIX 成组链接法

分配和回收的算法如下：

1. 分配一个空闲块

查L单元（空闲块数）内容：

当空闲块数>1 $i := L + \text{空闲块数}$ ；

从i单元得到一空闲块号；

把该块分配给申请者；

空闲块数减1；

当空闲块数=1 取出L+1单元内容（一组的第一块块号或0）；

若值=0，表示无空闲块，申请者等待；

若值不等于零，把该块内容复制到专用块；

该块分配给申请者；

把专用块内容读到内存L开始的区域。

2. 归还一块

查询L单元的空闲块数；

当空闲块数<100 空闲块数加1；

$j := L + \text{空闲块数}$ ；

归还块号填入j单元。

当空闲块数=100，则把内存中登记的信息写入归还块中；

把归还块号填入L+1单元；

将L单元置成1。



图里专用块L显示空块数20是目前只有20个空块，上限100！

分配空闲块时从下向上分配

2. UNIX 内存中所需数据结构

(1) 用户打开文件表

- 每个进程一个
- 维护打开文件的状态和信息
- 进程的PCB中记录了用户打开文件表的位置

文件描述符	打开方式	读写指针	i节点指针
...	...	...	...

(2) 系统打开文件表

- 整个系统一个，放在内存
- 用于保存已打开文件的FCB

FCB(i节点)信息	引用计数	修改标志
...	...	...

运行时文件结构

文件描述符

目录检查 — FCB ⇒ MEM

3. 文件操作的实现

1) 创建文件

实质是建立文件的 FCB

目的：建立系统与文件的联系

a) 在目录中为新文件建立一个目录项 (FCB/inode + 目录项), 根据提供的参数及需要填写有关内容

b) 分配必要的存储空间

2) 打开文件

a) 根据文件名在文件目录中检索, 并将该文件的目录项 (FCB) 读入内存, 建立相应的数据结构, 为后续的文件操作做好准备

b) 返回文件描述符/文件句柄

3) 部分文件操作函数

文件操作—建立文件

create (文件名, 访问权限)

① 检查参数的合法性

例如: 文件名是否符合命名规则;

有无重名文件;

合法→②, 否则→错误返回

② 申请空目录项, 并填写相关内容;

③ 为文件申请磁盘块; (FCB) 元数据 + 磁盘块

④ 返回

引导记录	超级数据块	空闲区管理	1-节点区	根目录区	文件和目录区
------	-------	-------	-------	------	--------

文件操作—读文件

read (文件句柄或文件描述符, 读指针, 要读的长度, 内存目的地址)

① 根据打开文件时得到的文件描述符, 找到相应的文件控制块 (目录项) 确定读操作的合法性

读操作合法→②, 否则→出错处理

问题: 文件尚未打开? 隐式/显式打开

② 将文件的逻辑块号转换为物理块号

根据参数中的读指针、长度与文件控制块中的信息, 确定块号、块数、块内位移

③ 申请缓冲区 (OS) 申请缓冲区 (磁盘块)

④ 启动磁盘I/O操作, 把磁盘块中的信息读入缓冲区, 再传送到指定的内存区 (多次读盘)

⑤ 反复执行③、④直至读出所需数量的数据或读至文件尾

文件操作—指针定位

seek (fd, 新指针的位置)

系统为每个打开文件维护一个读写指针, 即相对于文件开头的偏移地址 (读写指针指向每次文件读写的开始位置, 在每次读写完成后, 读写指针按照读写的数据量自动后移相应数值)

① 由fd查用户打开文件表, 找到对应的入口;

② 将用户打开文件表中文件读写指针位置设为新指针的位置, 供后继续写命令存取该指针处文件内容

思考

► 怎样实现系统调用rename (给文件重命名)? 路径 - 目录检索 - 找FCB - 改FCB

► 怎样实现系统调用copy (复制文件)?

► ...

... 找到FCB - 读入文件 (到buf...)

- 创建文件 - 写文件

copy
shell
open A
create B
open B
read A
write B
close A
close B

思考题 (cnm 课上给出的答案, 不保证准确性)

► 怎样实现系统调用 rename (给文件重命名) - open + 改 FCB (或 copy 到

命名为新名字的新建文件，然后删除旧文件)

- 怎样实现系统调用 copy (复制文件) - openA + create + openB + readA + writeB + closeA、closeB

4. 文件共享

1) 简介

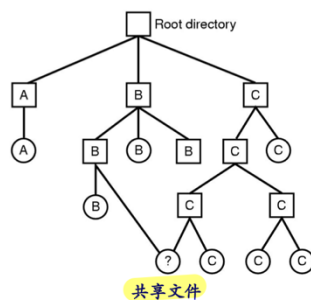
- a) 文件共享指一个文件被多个用户或进程使用
- b) 多用户系统中文件共享很必要

2) 一种实现方式：文件别名 (目的：节省时间和存储空间、交换信息)

a) 硬链接

- 利用多个路径名描述同一共享文件，多个目录项指向一个文件
- 路径名和目录项一一对应

例如：通过“**连接 (Link)**”命令，在用户自己的目录中对要共享的文件建立起相应的表目，即**建立两个文件的等价关系**



怎样实现? 怎样删除文件?

FCB中
FCB中直接给出文件地址?
问题: 删除文件

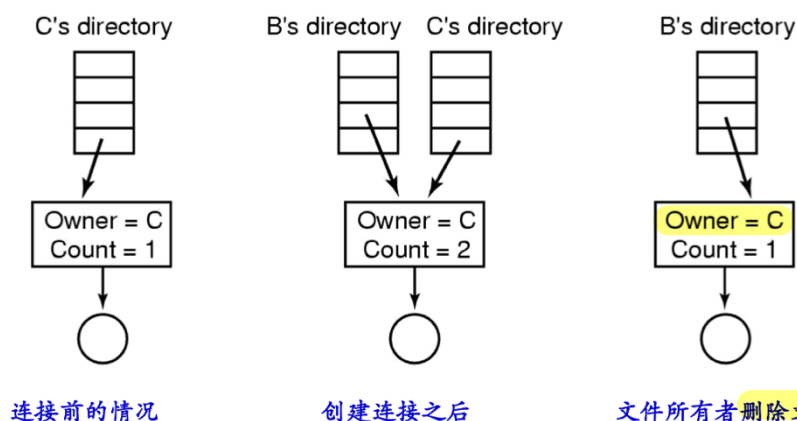
目录项指向I节点

以Linux为例：目录项分为两部分：文件名和索引结点；

通过多个文件名链接(link)到同一个索引结点，可建立同一个文件的多个彼此平等的别名；

别名的数目记录在索引结点的链接计数中，若其减至0，则文件被删除

思考：怎样删除文件？



b) 软链接

- 建立一种特殊类型的文件，通过文件内容与另一个文件连接
- 例子：“快捷方式”指向其他文件

又称 符号连接、快捷方式

- ▶ 建立一种特殊类型 (Link) 的文件，其内容是要共享的文件路径名
- ▶ 只有真正的文件所有者才有指向i节点的指针
- ▶ 可以建立任意的别名关系，甚至原文件是在其他计算机上

问题：系统开销大；目录结构可能形成环状

优势：计算机网络环境下可用

5. 挂载 (mount) 和卸载 (unmount)

- 1) 挂载：将一个文件系统加入到另一个文件系统
- 2) 用户提供被挂在的文件系统的根目录/挂载点

重点小结

1. 基本概念
 - 1) 文件、文件系统、文件目录
 - 2) 文件类型
 - 3) 文件的逻辑结构
2. 文件系统实现
 - 1) 文件系统布局
 - 2) 文件的物理结构
 - 3) 文件目录的实现
 - 4) 内存结构
 - 5) 文件操作
 - 6) 磁盘空间管理
 - 7) 文件共享
3. 文件系统实例
 - 1) UNIX
 - 2) FAT16/32
 - 3) NTFS